

Kasbana — Git, Push & Infrastructure Flow

How the monorepo distributes, how CI gates code, and how a push reaches production on EC2.

Django + DRF backend

GitHub Actions

Docker · Caddy · Postgres · Redis

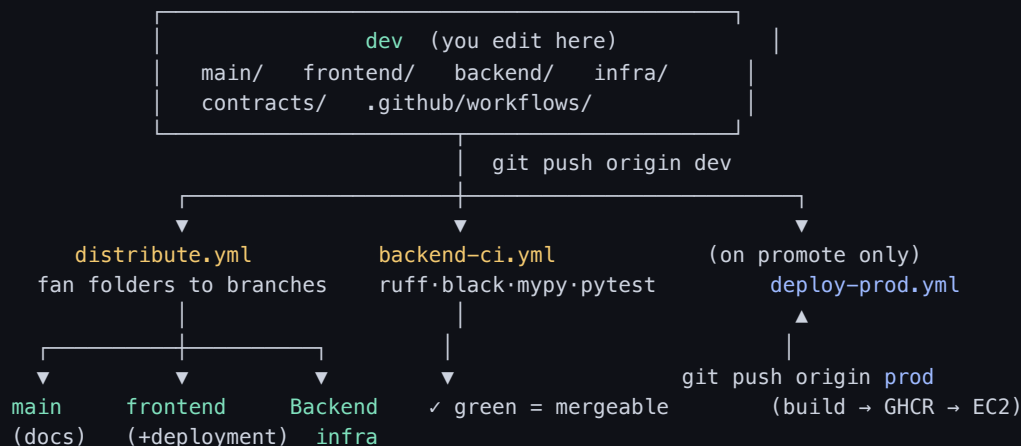
AWS EC2 (t3.small)

Repository: github.com/momenawab/Kasbana · Generated 2026-06-28 · Owner: Momen · Partner: Joe

1. The big picture

Kasbana is a **monorepo**. You only ever edit one branch — **dev**. Everything else is automated: pushing **dev** fans each top-level folder out to its own branch, runs the backend test suite, and (when you promote to **prod**) builds a Docker image and rolls it onto the server.

One source of truth → many automated outputs



2. Branch model

Branch	Holds	From folder	Type
dev	The whole project — the only branch you edit	—	SOURCE
main	Docs / project overview	main/	GENERATED
frontend	React app source	frontend/	GENERATED
Backend	Django API source	backend/	GENERATED
infra	DevOps: Docker, Caddy, deploy scripts	infra/	GENERATED

deployment	Built static site (served by Hostinger)	built frontend/dist	GENERATED
prod	Release gate for the backend → EC2	promoted from dev	DEPLOY

Golden rule: edit only `dev` . The generated branches are overwritten on every push (`force_orphan`) — never commit to them directly. Shared areas (`core/` , `common/` , `contracts/`) change only via reviewed PR.

3. The everyday push flow (dev)

This is what happens on every normal change. **No deploy** — only distribution + CI.

```
git add -A
git commit -m "your message"
git push origin dev
```

Two workflows fire in parallel on a dev push

```
push dev
├─┬─ distribute.yml
│  │   ├── main/      → branch main
│  │   ├── frontend/  → branch frontend
│  │   ├── backend/   → branch Backend
│  │   ├── infra/     → branch infra
│  │   └─ build FE    → branch deployment → Hostinger serves it
└─┬─ backend-ci.yml (only if backend/ or contracts/ changed)
   │   ├── ruff (lint)
   │   ├── black (format check)
   │   ├── mypy (type check)
   │   ├── manage.py check
   │   ├── makemigrations --check (no missing migrations)
   │   └─ pytest ✓ green = safe to promote
```

All distribution pushes use `GITHUB_TOKEN`, so they never re-trigger each other (no loops).

4. The release flow (dev → prod)

Production deploys are **deliberate**. You promote a known-good `dev` to `prod`; only then does the se

```
git checkout prod
git merge --ff-only dev      # fast-forward: prod == dev
git push origin prod        # ← triggers deploy-prod.yml
git checkout dev            # back to working branch
```

deploy-prod.yml – build in CI, the 2GB box only pulls

push prod



```
job: build
  docker build (context: backend/)
  Dockerfile: infra/docker/Dockerfile
  push image → ghcr.io/momenawab/
               kasbana-backend:<sha> + :latest
```



```
job: deploy (SSH to EC2)
  rsync infra/ → /opt/kasbana/infra
  docker login ghcr.io
  scripts/deploy.sh:
    docker compose pull
    manage.py migrate
    docker compose up -d
    force-recreate caddy (apply Caddyfile)
```



live ✓ api.kasbana.net / admin.kasbana.net

Why build in CI, not on the box? The t3.small has only 2 GB RAM – a docker bui

5. Production infrastructure (AWS EC2)

Host

Item	Value
Instance	EC2 t3.small (2 vCPU / 2 GB)
OS	Ubuntu 24.04 LTS
Public IP	13.49.70.197
Region	eu-north-1 (Stockholm)
Swap	4 GB (OOM safety)
App dir	/opt/kasbana/infra
Registry	GHCR (ghcr.io)

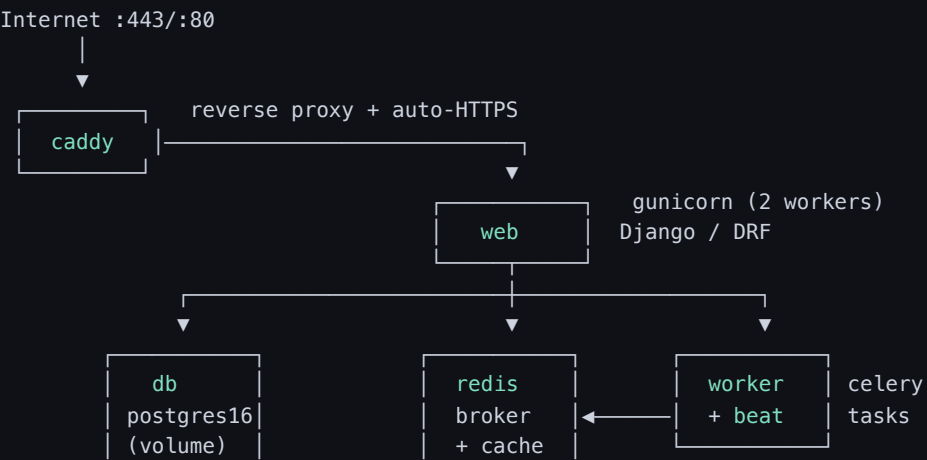
Domains & TLS

Host
api.kasbana.net
admin.kasbana.net
kasbana.net

Caddy auto-provisions Let's Encrypt certificates

The container stack (docker compose)

compose.prod.yml – one box, six services



```
nightly pg_dump → /opt/kasbana/backups (backup.sh)
```

Image build (infra/docker/Dockerfile)

- Base `python:3.12-slim`; deps installed from `requirements.txt` (layer-cached).
- `collectstatic` at build time → WhiteNoise serves static; Caddy stays a pure proxy.

```
Runs as a non-root user: gunicorn config.wsgi --bind :8080. Worker/heat override the co
```

Secrets – where they live

Secret	Stored in
SSH key (.pem)	GitHub Actions secret <code>EC2_SSH_KEY</code>
<code>EC2_HOST</code> / <code>EC2_USER</code>	GitHub Actions secrets
App env (SECRET_KEY, DB pw, wallet certs...)	<code>/opt/kasbana/infra/.env</code> on the box
GHCN auth	<code>GITHUB_TOKEN</code> (per-deploy)

The `.pem` and the server `.env` are **never** committed. Secrets were generated

6. Repository layout (inside `dev`)

```
Kasbana/ (dev branch)
├─ main/                docs & project plan          → main branch
│   └─ docs/            contract, briefs, onboarding
├─ frontend/           React (Vite) bilingual site → frontend / dep
├─ backend/            Django API                  → Backend branch
│   ├── config/         settings (base/dev/prod), urls, celery
│   ├── core/ [FROZEN]  models · enums · ledger · auth · tenancy
│   ├── common/         errors · pagination · permissions
│   ├── wallets/ billing/ service & entitlement seams
│   └─ tests/           pytest suite
├─ infra/              DevOps                      → infra branch
│   ├── docker/Dockerfile
│   ├── compose.prod.yml
│   ├── caddy/Caddyfile
│   └─ scripts/         provision.sh · deploy.sh · backup.sh
├─ contracts/openapi.yaml frozen v1 API contract
└─ .github/workflows/   distribute · backend-ci · deploy-prod
```

7. Ownership

Owner	Modules / branches
Momen	<code>core</code> , <code>common</code> , <code>config</code> , <code>enrollment</code> , <code>wallets</code> , <code>billing</code> , a
Joe	<code>loyalty</code> , <code>dashboard</code>
Shared (PR only)	<code>core/</code> , <code>common/</code> , <code>contracts/openapi.yaml</code>

8. Command cheat-sheet

Daily work

```
# normal change (CI only)
git add -A
git commit -m "msg"
git push origin dev

# run the gate locally first
cd backend && source .venv/bin/activate
ruff check . && black . && mypy .
pytest
```

Release to production

```
git checkout prod
git merge --ff-only dev
git push origin prod # deploy
git checkout dev
```

On the server

```
ssh -i key.pem ubuntu@13.49.70.19
cd /opt/kasbana/infra
docker compose -f compose.prod.yml up
docker compose -f compose.prod.yml
```

9. New teammate onboarding

```
git clone https://github.com/momenawab/Kasbana.git
cd Kasbana
./scripts/onboard.sh # mac/Linux/WSL (Windows: scripts/o
```

Sets up the backend venv + deps + `.env`, runs migrations and tests, installs the front

10. Live endpoints

URL	Purpose
<code>https://api.kasbana.net/api/health/</code>	Health check
<code>https://api.kasbana.net/api/docs/</code>	Swagger / OpenAPI UI
<code>https://admin.kasbana.net/admin/</code>	Django admin panel